

Laboratorio Nro. 5

Introducción a RTOS

Comisión Tauro - Aguirre

Actividad 1: Introducción a FreeRTOS

Descripción del funcionamiento

El programa implementa el parpadeo de un LED a una frecuencia de 1 Hz utilizando el kernel FreeRTOS.

Para lograrlo, se crean dos tareas:

- **vTaskTurnOn:** enciende el LED y espera 500 ms.
- **vTaskTurnOff:** apaga el LED y espera otros 500 ms.

Cada tarea se sincroniza mediante semáforos binarios, garantizando que las acciones de encendido y apagado ocurran de manera alternada.

Inicialmente, se entrega con *xSemaphoreGive* el semáforo de la tarea *TurnOn*, de modo que el ciclo comienza con el LED apagado.

Una vez encendido el LED, la tarea libera el semáforo de la tarea *TurnOff*, que luego realiza la operación contraria, repitiendo el proceso.

Mecanismo de sincronización

Se utilizan dos semáforos (*xLedSemaphore1* y *xLedSemaphore2*) creados con *xSemaphoreCreateBinary()*. Cada tarea se bloquea en *xSemaphoreTake()* hasta que la otra le cede el turno. Este método evita que ambas tareas accedan simultáneamente al mismo recurso (el LED) y mantiene el ritmo de parpadeo requerido.

Frecuencia del parpadeo

Cada tarea espera 500 ms mediante *vTaskDelay(pdMS_TO_TICKS(500))*, logrando un ciclo total de 1 s entre encendido y apagado, es decir, una frecuencia de 1 Hz.

Estados de las tareas en FreeRTOS

Durante la ejecución, las tareas atraviesan los siguientes estados:

- **Running:** cuando el planificador ejecuta la tarea.
- **Blocked:** mientras esperan la obtención de un semáforo o durante *vTaskDelay()*.
- **Ready:** cuando están listas para ejecutarse, esperando que el planificador las seleccione.
- **Suspended o Deleted:** no se utilizan en este programa, pero representan tareas pausadas o eliminadas.

Actividad 2: Mediciones de luminosidad por I2C

Descripción

El programa implementa el parpadeo de dos LEDs utilizando timers de software del módulo *timers.h* de FreeRTOS.

Cada LED se asocia a un temporizador independiente, que ejecuta una función callback al cumplirse su período:

- LED1 (pin 12): parpadea a una frecuencia fija de 4 Hz
- LED2 (pin 11): parpadea inicialmente a 1 Hz, pero su frecuencia puede modificarse a través del puerto serie.

El usuario puede ingresar los valores:

- '1' → LED2 cambia a 0.5 Hz (toggle cada 1000 ms).
- '2' → LED2 cambia a 2 Hz (toggle cada 250 ms).

Los *timers* se crean mediante *xTimerCreate()* y se inician con *xTimerStart()*. Cada callback realiza un toggle del LED correspondiente utilizando *digitalWrite()* y *digitalRead()*.

Funciones de callback

- **vTimer1Callback:** se ejecuta periódicamente para el LED1, cambiando su estado con una frecuencia fija de 4 Hz.
- **vTimer2Callback:** controla el LED2, y su período puede modificarse dinámicamente mediante *xTimerChangePeriod()* según la entrada por serial del usuario.

Ventajas del uso de timers de software

El uso de timers de software permite ejecutar tareas periódicas sin crear tareas adicionales ni utilizar semáforos. Esto reduce el uso de memoria RAM y la carga del planificador de tareas, haciendo el programa más eficiente y simple.

Los timers son gestionados por una tarea interna del sistema operativo que garantiza mayor precisión temporal y un control confiable de los intervalos de ejecución.

Decisiones de diseño

- Se optó por el uso de *pdTRUE* en la creación de los timers para habilitar el modo auto-reload, de forma que el parpadeo sea continuo.
- Los períodos de los timers se definen en milisegundos y se convierten a ticks con *pdMS_TO_TICKS()*.
- Se inicializa la comunicación serial para permitir la modificación dinámica de la frecuencia del segundo LED.